PRODUCT SPEC + TECHNICAL SPEC · V2 · OCT 13, 2026

Tesla Lab

Take a wild tech claim seriously. Let the physics decide. Design the part that works, then find the missing piece in the world's patents.

One line for judges: “We took the conspiracies seriously. The math gave the verdict, designed a product from the part that survived, and searched 100+ years of patents, even in other industries, for the piece that was missing.”

Event: Nebius × NVIDIA Global AI Hackathon (nebiusglobalaihackathon.devpost.com)
Deadline: Fri Oct 30, 10:00am PDT = **Sat Oct 31, 1:00am Malaysia**. Plan to submit Thu Oct 29.
Builder: Chew · **Code today:** starter repo v3 (82 tests pass in mock mode)

What changed since spec v1: the Design Engine, Innovation Loop, Tesla Archive and 20-claim eval are now **BUILT** (v2). OpenMath-Nemotron as the math consultant is **BUILT** (v3). **NEW:** the Patent Search Planner (Find · Check · Improve) and the 💡 Wildcard step, both **PLANNED** (not in code yet). Nothing has run live yet: no API keys or GPU are connected.

Tags: BUILT in the v3 repo, tested in mock mode · NEW added in this spec version · PLANNED designed, not built · UNVERIFIED not confirmed live or at a primary source · EST. estimate · Verdicts: WORKS MISSING PIECE BREAKS PHYSICS

Contents

The Innovation Loop 2
Product spec · problem, users, use cases 3
Patent Search Planner + 💡 Wildcard (product) 4
Core flows and screens 6
Demo, what judges see, metrics, scope 7
Tech spec · architecture 8
Product Design Engine 9
Math checker (OpenMath-Nemotron) 10
Tesla Archive and gap search 11
Patent Search Planner (tech) 11
💡 Wildcard (tech) 13
Data model and JSON 14
Models, cost, errors, testing 14
Judging map and build schedule 15
What's verified, what isn't 17

The Innovation Loop

Debunking is half the product. The other half is the question every inventor asks next: **so what could work, and what's stopping it?**

1. Design the part that works BUILT
The WORKS claim becomes a product: a vetted design calculator sizes it, checks safety, prices parts and writes the render prompt from the numbers.

2. MISSING PIECE → "what would need to be true" BUILT
For each MISSING PIECE, Nemotron writes 2–4 requirements with a measurable target. Tavily + arXiv find who is working on each today, and a transparent score ranks them.

3. Find the missing piece in patents NEW PLANNED
The Patent Search Planner searches by **function**, not product name, across industries, follows citation chains and reads claims. The 💡 Wildcard step borrows the same physical principle from a totally unrelated niche.

4. Speed
A first pass with checked math in about 1–3 minutes per idea EST., against days to weeks by hand EST.. It shows its work; it doesn't replace an engineer or a patent attorney.

Worked example: Wardenclyffe (repo output, mock mode)

Step	What Tesla Lab says
Verdicts	Short-range resonant coils WORKS (18% at 2 m). Power around the planet MISSING PIECE (Earth–ionosphere cavity Q ≈ 5). "Free" power BREAKS PHYSICS.
Design	Tesla Shelf: 10 W at 0.5 m, 63% coil to coil, keep people ~0.85 m off the coil axis at full power.
Math checker	OpenMath re-derives each equation; badge "agreed / DISAGREED" vs Python (canned in mock mode).
What would need to be true	Beamed power > 30% DC-to-DC over 1 km (feasibility 0.45) · guided surface wave > 50% at 10 km, kW scale (0.12) · cavity Q > 1000 at 7.8 Hz (0.02). Each with live-captured papers and articles.
Patents PLANNED	Planner: "deliver power across a room WITHOUT exceeding field limits" → CPC classes for wireless power, citation chain from Tesla's US 787,412 (its Google Patents page lists hundreds of citing documents).

Feasibility numbers are the model's judgment (canned in mock mode); the gap leads were captured from the live Tavily and arXiv APIs on Oct 13 and are snippets, not read papers.

Problem

Fringe-tech claims spread faster than anyone can check them, and the checks that exist are opinions without math. Real R&D teams have the mirror-image problem: they are mid-project, stuck on one conflict (“less backpressure, but no extra noise”), and the answer is often already sitting in a patent from another industry, written in words they would never search for.

Tesla Lab gives each idea a verdict per sub-claim with computed numbers and sources, designs the part that works, and goes looking for the missing piece across the world's patents.

Target users

Who	Their job	What they get
R&D teams NEW	Get unstuck mid-project; vet ideas before spending months	Find / Check / Improve searches with a reasoning trail, claim-level fit, legal-status flags, Wildcard ideas
Makers & hardware founders	Find what's buildable and what it takes	Spec sheet, parts list and cost, R&D roadmap for the missing piece
Educators & students	Turn a viral claim into a lesson	Verdict cards, the equation, the code, step-by-step derivation
Journalists & skeptics	Check a claim with primary sources	Citations down to document and page

Use cases NEW

R&D teams rarely start from zero; they are usually mid-research and stuck. Three plain-word modes, one engine:

🔎 Find PLANNED

The team is mid-research and stuck. **Start from the goal + what failed.** Emphasis: cross-industry classification search (step 3); the missing piece is usually in another field. Output: ranked mechanisms from other industries, each with the patent, the claim text that fits, and why.

✅ Check PLANNED

Validate an idea before spending months. **Start from the idea.** Emphasis: narrow and deep in the idea's own class, read the claims, filter by date. Output: “closest prior art” list with claim overlap, legal-status flags (not legal advice) plus the physics verdict from the existing pipeline.

🔁 Improve PLANNED

Each test result adds a constraint. **Start from the last run + new result.** Emphasis: re-run with the new constraint, show what changed, watch for new filings. ROADMAP ONLY a closed loop with 3D-printed prototypes and test data flowing back in automatically.

💡 Wildcard PLANNED

An extra step in any mode. Strip the problem to its bare physical principle and search niches that have nothing to do with it: musical instruments, heart valves and biology, nature, toys, kitchen tools, other CPC sections. Returns “why not try this?” ideas ranked by **surprise × does-the-math-work**. The move a human expert usually misses, and the judges' wow moment.

Patent Search Planner NEW PLANNED

The core rule: **search by function, not by product name.** Patents use their own language (“fluid conduit”, not “pipe”), and the best answer is often in a different industry. The agent runs five steps and shows every one in a reasoning trail.

> **1. Abstract** → a function plus a conflict. `reduce backpressure WITHOUT increasing noise`

> **2. Expand** → synonyms and patent-speak for every term: pipe → fluid conduit, duct, passage; noise → acoustic, sound attenuation, silencing.

> **3. Classify** → the CPC codes for that function, then the same function in other industries' codes.

> **4. Chain** → follow citations backwards (older roots) and forwards (newer improvements).

> **5. Read claims** → score each hit on how well it solves your conflict; flag expiry / legal status (not legal advice).

> 💡 **Wildcard** → bare principle (“smooth pressure pulses in a flowing gas”) → unrelated niches → “why not try this?” ranked by surprise × math.

Same engine, different emphasis

Mode	Starts from	What the planner leans on
Find	Goal + what failed	Step 3 wide: home class and 3–6 cross-industry classes; failed approaches become exclusions
Check	The idea	Narrow and deep in its own class; step 5 on every hit; date filter (anything before the team's date is prior art); non-patent literature on

Improve	Last run + new test result	Add the constraint, re-run, show “new / dropped / re-ranked”; a watch for filings newer than the last run

Example: an exhaust team (Find mode)

Input	“Our new muffler cut noise but backpressure went up 15%. Tried bigger pipe: too loud.”
Abstract	Function: attenuate pressure pulses in exhaust gas. Conflict: lower backpressure WITHOUT more noise. Exclude: plain diameter increase.
Classify	Home: **F01N 1/02** silencing by resonance (incl. F01N 1/023 Helmholtz resonators), **F01N 1/06** interference. Cross-industry: **F16L 55/04** damping pulsations in fluids (pipelines), **G10K 11/172** noise damping using resonance (general acoustics).
Chain + claims	Top hits' cited and citing patents; each scored against the conflict with the quoted claim line.

CPC titles checked on the USPTO CPC scheme pages (Oct 13). The input and the walk-through are illustrative; no search has been run yet.

💡 Wildcard on the same problem

Principle: smooth 100 Hz pressure pulses in hot flowing gas (4-cylinder engine at 3,000 rpm; ~400 °C exhaust → speed of sound ≈ 510 m/s). Sizes computed in Python on Oct 13 with those assumptions; the ideas are illustrative, not yet searched.

Niche	Why not try this?	Math
Music (G10D 7/00 wind instruments)	A side tube a quarter-wavelength long cancels the pulse; coil it like a French horn to fit under the car	1.28 m at 100 Hz (0.64 m at 200 Hz): fits coiled → **high**
Biology (heart)	The aorta stretches on each heartbeat and smooths the pulse (Windkessel effect): a compliant chamber	Needs a sizing calc; hot gas limits elastic materials → **medium**
Acoustics (G10K 11/172)	Helmholtz resonator	8.8 L for a 40 × 60 mm neck: bulky; and already in the home class (F01N 1/023) → **low surprise**
Nature (owl feathers)	Serrated edges quiet airflow	Targets broadband flow noise, not 100 Hz pulses → “fun, but the numbers don't fit”

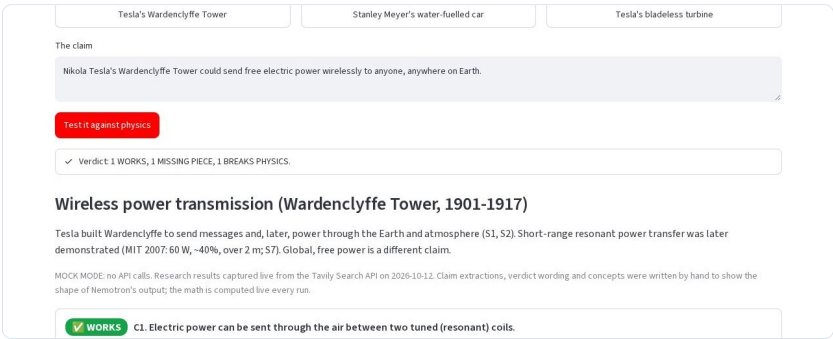
What judges see (planner scene, ~40 s of the video)

1. A real-sounding R&D problem typed in Find mode; the trail shows the conflict line and the patent-speak terms.
2. CPC codes light up: the home class, then two other industries. Hits stream in with a quoted claim line each.
3. A citation chain grows from one 1990s root patent to recent filings.
4. 💡 **Wildcard:** “why not a French-horn-coiled tuning tube?” with the 1.28 m number computed live and the math checker badge.
5. One line: “Today it finds and checks. Tomorrow it iterates with your 3D-printed prototypes.” (roadmap)

Core flows

Flow	Steps	Status
A. Check a claim	Type a claim or tap an example → live progress → verdict cards with math, inputs, sources, math-checker badge	BUILT
B. Design from WORKS	Spec card: specs, chart, safety check, parts list, the code → download Markdown / PDF / JSON → render prompt from the numbers	BUILT
C. Explore the gap	MISSING PIECE → ranked “what would need to be true” with live leads	BUILT
D. Dig the archive	Matching patent pages cited “US 787,412, page 5” with a snippet	BUILT
E. Find / Check / Improve	Pick a mode → describe goal, failure or idea → 5-step trail → ranked patents with claim fit and status flags → open on Google Patents	PLANNED
F. 💡 Wildcard	Tap “Wildcard” on any planner run → principle → far-niche ideas with a computed number each	PLANNED
G. From verdict to planner	On a MISSING PIECE card, “Search patents for this” prefills Find mode with the requirement as the goal	PLANNED
H. Take it away	Report JSON + spec sheet PDF BUILT · shareable run page PLANNED	

Screens



The real app today (Streamlit, mock mode).

S1–S4 Home, progress, verdict, concept	BUILT Mode banner, examples, step log, verdict cards (equation, chart, code, inputs, sources), product concept and render.
S5 Spec card	BUILT Tabs: specs · chart · safety · BOM · code; Markdown / PDF / JSON download.
S6 Innovation loop	BUILT Per MISSING PIECE: ranked requirements with target, feasibility, leads.
S7 Math checker	BUILT Panel + per-verdict badge and “Step-by-step derivation”.
S8 Archive	BUILT Matching pages with document + page.
S9 Patent planner	PLANNED Mode tabs (Find / Check / Improve), input box, the 5-step trail as expanders, hit cards: title, CPC, date, status flag, claim quote, fit score, “found via”.
S10 Wildcard	PLANNED Orange cards: niche, “why not try this?”, the number, surprise and math badges.

Demo claims

All three run end to end in mock mode with real math (repo, Oct 13).

Claim	Verdicts	Product
Wardenclyffe tower	Coils WORKS (18% at 2 m, <1% by 3.6 m) · Earth as conductor MISSING PIECE (cavity Q ≈ 5, rings down in ~0.1 s) · free power BREAKS PHYSICS	Tesla Shelf
Meyer's water car	Water as fuel BREAKS PHYSICS (loop returns 0.13 of the input) · cheaper electrolysis BREAKS PHYSICS (1.229 V minimum) · hydrogen as storage WORKS (~33% round trip)	SunCell Backup
Tesla turbine	Disc drag WORKS (36–41% measured) · 97% MISSING PIECE · replace power-plant turbines MISSING PIECE (~2.2× the steam)	DiscFlow Pump



Tesla Shelf



SunCell Backup



DiscFlow Pump

What judges see (3-minute video)

1. Tesla's 1901 promise, cited from his own patent (archive, page-level).
2. Sources stream in; Nemotron 3 Super lists the testable claims.
3. Python computes; three verdict cards land; the guardrail overrules a too-kind answer; **“math checker agreed”** badges from OpenMath-Nemotron.
4. The WORKS card becomes the Tesla Shelf spec: 10 W at 0.5 m, 63% coil to coil, and the honest safety pushback (keep ~0.85 m off the coil axis).
5. Side by side: a plain chatbot gets a number wrong; Tesla Lab computes it (eval set).
6. PLANNED The MISSING PIECE goes to the patent planner, then the 💡 **Wildcard** moment (see the planner pages).
7. Under the hood: Tavily → Nemotron on Token Factory → Python sandbox ⇄ OpenMath on a Nebius GPU → guardrail.

Success metrics (targets)

Verdict accuracy	≥ 90% on the 20-claim eval set (live run), and beat the chatbot baseline
Traceability	100% of numbers carry a source id or “assumed”
Math checker	Agreement rate reported per run; every DISAGREED gets a human look
Honesty	0 of 20 red-team “free energy” prompts get WORKS on the free-energy part
Planner PLANNED	Every hit shows a claim quote that exists on its page; ≥ 1 cross-industry hit per Find run; ≥ 1 Wildcard idea per demo problem passes its math
Speed	Median live verdict run under 90 s; planner run under 3 min EST.

Out of scope: building or certifying hardware · legal opinions (freedom-to-operate, validity) · claims outside physics and engineering · user accounts · full 3D CAD and the 3D-print loop (roadmap only).

PART 2 · TECHNICAL SPEC

Architecture

0. Input BUILT Claim text from the UI or CLI (`python -m tesla_lab`).

▼

1. Research BUILT Tavily: 3 searches (primary source, modern work, debunks) + extract top 3 → S1…Sn; Tesla Archive hits → A1…An, interleaved by rank.

▼

2. Extract BUILT Nemotron 3 Super → 2–4 testable claims with quantities (source id or "assumed") and a calc spec (vetted template or custom code).

▼

3. Compute BUILT 7 vetted calculators in-process; model-written SymPy/NumPy in the sandbox (AST check + isolated subprocess). Python is the **calculator of record**.

▼

4. Verdict + guardrail BUILT Nemotron explains; `apply_guardrails()` forces BREAKS PHYSICS on a conservation / second-law flag and caps at what the numbers support.

▼ then in parallel

5a. Gap search BUILT
Requirements → Tavily + arXiv → ranked.

5b. Design Engine BUILT
WORKS → ProductSpec, BOM, safety, render prompt.

▼

6. Math checker BUILT Per claim + spec: OpenMath-Nemotron derives, Python's number is compared → agreed / DISAGREED / inconclusive / skipped.

▼

7. Patent Search Planner NEW PLANNED Find / Check / Improve: Abstract → Expand → Classify → Chain → Read claims, then the 💡 **Wildcard** branch with its math sent back through steps 3 + 6. Runs from a MISSING PIECE card or on its own.

▼

8. Report BUILT One pydantic `Report`; the UI renders only from it; JSON export. PLANNED `search_plan` field.

Repo map (v3)

BUILT `research.py` · `llm.py` · `prompts.py` · `calcs/` (7) · `sandbox.py` · `pipeline.py` · `design/` (wpt, disc_pump, h2_storage) · `spec_sheet.py` · `gaps.py` · `archive.py` + `archive/` (6 patents, prebuilt index) · `mathcheck.py`, `openmath.py`, `latexmath.py` · `deploy/openmath/` · `evals/` + `scripts/run_eval.py` · `app.py`.
PLANNED `patents/`: `planner.py` (5 steps + modes), `cpc.py` (CPC title table + distance), `gpatents.py` (patent-page reader + query links), `bq.py` (optional BigQuery), `wildcard.py`, `mock/planner_exhaust.json`.

Product Design Engine BUILT

A design calculator takes requirements and returns a `ProductSpec` in which every number is tagged **req**, **calc**, **assumed** or **est.**

Calculator	Sizes

wireless_power	Coil turns, L, C, capacitor voltage, Q (copper + radiation + capacitor ESR), k(d) from exact coaxial-loop mutual inductance, η(d), amplifier power, coil current, ICNIRP exposure screen
disc_pump	Disc diameter, gap from boundary layer √(ν/ω), disc count, motor, running cost vs centrifugal
h2_backup	kg H_2, tanks, electrolyser kW, extra PV, water, round trip, battery cost crossover

Tesla Shelf (repo output)

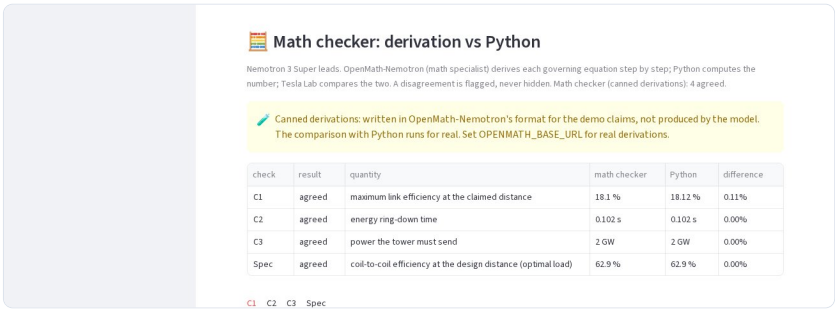
Target	10 W into the load at 0.5 m, 6.78 MHz
Tx coil	0.5 m, 2 turns, 6 mm copper tube, 4.79 µH, 115 pF (727 V rms → spec ≥ 2.5 kV), Q ≈ 717
Rx coil	0.15 m, 6 turns, 9.07 µH, 60.8 pF, Q ≈ 590
Link	k = 0.0066, $k\sqrt{(Q_1Q_2)} = 4.28$ → **62.9%** coil to coil; halves by 0.75 m; <10% beyond 1.01 m
Power	Amplifier 15.9 W; wall ≈ 18.7 W (85% amplifier assumed)
Exposure screen	On-axis H at 0.5 m 1.275 A/m vs ICNIRP 2020 whole-body 0.324 A/m: **fail** at full power; passes beyond ~0.85 m or below ~0.65 W
BOM	~$380 prototype EST.

ICNIRP 2020 Tables 5–6 and 2010 Table 4 values were checked against the PDFs (Oct 13). The screen is approximate: on-axis peak field vs body-averaged reference levels. Capacitor Q, proximity factor and amplifier efficiency are assumptions; BOM prices are estimates. The render prompt is written from the computed numbers.

Math checker: OpenMath-Nemotron BUILT

Role	Model	Where
Lead engineer	Nemotron 3 Super	Token Factory, pay per token
Math consultant	OpenMath-Nemotron (7B rec.)	Chew's own Nebius GPU VM, vLLM, OpenAI-compatible
Calculator of record	Python (SymPy / NumPy)	The app + sandbox

1. For each claim and the spec, a math problem is written with the **same inputs Python used**: by code for the 7 calculators and 3 designers, by Nemotron 3 Super for custom-code claims.
2. OpenMath derives step by step and boxes the answer (prompt and temperature 0.6 per NVIDIA's model card; max 12,000 tokens).
3. The boxed answer (number or formula, parsed with SymPy) is compared with Python: **agreed** within 2% (×100 / ×1000 unit slips noted), **DISAGREED** in red and added to the notes, **inconclusive**, **skipped** or **error**. A disagreement never changes the verdict.



Math checker panel, mock mode. Demo derivations are **canned** (hand-written in the model's format, labelled) until the GPU endpoint exists; the comparison runs for real.

GPU and cost EST.

`deploy/openmath/`: `create_vm.sh` (Nebius "Host an LLM" quickstart, Ubuntu 24.04 CUDA 12, region eu-north1), `serve.sh` (vLLM, bf16, 16k context), `start_vm.sh` / `stop_vm.sh`, `idle_stop.sh`, `scripts/check_openmath.py`. **7B on 1× L40S ≈ $1.55/h → ≈ 64 h for the $100 credit** (Nebius price pages, Oct 13; whether the credit covers disk and IP is unchecked). One check may take 1–3 min on an L40S EST.. Stop with `stop_vm.sh`, never `sudo shutdown` (Nebius docs: it restarts and keeps billing). UNVERIFIED nothing has run on a real GPU.

Tesla Archive and gap search BUILT

Archive: 6 public-domain US patents from Google Patents (US 645,576 · 649,621 · 787,412 · 1,119,732 · 1,061,206 · Meyer's 4,936,961), split by page, cleaned, chunked (~900 chars, 150 overlap), BM25 by default or hybrid with embeddings (local, or Token Factory `/v1/embeddings` UNVERIFIED with a key). Hits cite "US 787,412 ..., page 5". FBI Vault (403 to scripts) and CIA reading room PDFs are added by hand. PLANNED quote-match check on every citation.

Gap search: for each MISSING PIECE, 2–4 requirements (target, feasibility 0–1, web + arXiv query). Tavily (`time_range=year`) + arXiv API (≤ 1 request / 3 s). `evidence = Σ relevance × 0.5^(age_years/2)`; `score = 0.6 × feasibility + 0.4 × min(1, evidence/3)`. The planner reuses this scoring style and its leads.

Patent Search Planner: tech NEW PLANNED

Algorithm

Step	How	Output
1 Abstract	Nemotron 3 Super, JSON mode. Find: goal + what failed; Check: the idea; Improve: last plan + new result	`function`, `conflict{improve, without}`, `principle`, `exclude[]`
2 Expand	Super: 3–6 synonyms + patent-speak per term; builds query strings in Google Patents syntax	`terms[]`, `queries[]`

3 Classify	Super proposes CPC codes; each is checked against a local CPC title table (unknown codes dropped). Cross-industry: match the function words against CPC titles in other sections/classes	`cpc[]` tagged home / cross
4 Chain	Top 5–10 hits: backward ("Patent Citations") and forward ("Cited By") from each patent page, or BigQuery for bulk; 1 hop, 2 if time	hits tagged `found_via`
5 Read claims	Fetch claims; Super scores fit to the conflict 0–1 and must quote the claim line (quote string-matched to the page, else fit = 0); read legal status + dates from the page	`fit`, `quote`, `status_flag`

Ranking (initial weights, tune on the demo problems): `rank = 0.5·fit + 0.2·cpc_match + 0.2·log-scaled forward citations + 0.1·recency`. Check mode puts date first (everything older than the idea is prior art); Improve sorts "new since last run" to the top. The trail stores what each step did, kept and dropped.

Google Patents features used (checked Oct 13)

- **Query syntax** (Google Patents Help, "Searching", support.google.com/faqs/answer/7049475): `cpc:A01B` or a bare code; a CPC search includes all child codes; `CPC=B60R22/low` ; field search `TI=` `AB=` `CL=` ; `before:2010` `after:"Jan 2006"` (filing date, switchable to priority/publication); `country:US` `status:grant` `assignee:` `inventor:` ; quotes, minus to negate; `NEAR/x` , `ADJ/x` , `WITH` , `SAME` (these change ranking only, not retrieval); keywords are auto-stemmed with close synonyms; "Include non-patent literature" adds Google Scholar; the Prior Art Finder suggests terms from a pasted paragraph.
- **Patent pages** (checked on patents.google.com/patent/US787412A): claims, CPC classifications, "Patent Citations" (backward), "Cited By" (forward, examiner vs third-party marked), "Similar Documents", and "Legal status" (shown there as "Expired - Lifetime") with Google's note that it "is an assumption and is not a legal conclusion".
- **robots.txt** (patents.google.com/robots.txt): `Disallow: /*` except `/patent/` , `/advanced` , `/sitemap/` . So the app **reads individual patent pages** but does **not** scrape the search results page; it gives the user clickable Google Patents query links instead.

Where hits come from

Source	Use
Tavily, `include_domains: ["patents.google.com"]`	Keyword discovery of patent pages, live (parameter in Tavily's search docs; hit quality for patents UNVERIFIED)
Google Patents Public Datasets on BigQuery	CPC-filtered and citation queries in bulk (below). Optional: needs a Google Cloud project
Patent pages	Claims, citations, status for each shortlisted hit (allowed by robots.txt)
Tesla Archive + arXiv	Already built; non-patent prior art

USPTO CPC scheme pages	CPC titles for the local table (e.g. uspto.gov/…/cpc-F01N.html) UNVERIFIED best bulk format to load

BigQuery: bulk and citation access

Exists (Google Cloud blog, Oct 31, 2017; github.com/google/patents-public-data table docs): `patents-public-data.patents.publications` (IFI CLAIMS + Google, CC BY 4.0: worldwide bibliographic data + US full text) has `cpc[]`, `citation[]` (backward), `title_localized`, `abstract_localized`, `claims_localized` (US only), dates; `google_patents_research.publications` adds `cited_by[]` (forward), `similar[]`, `top_terms`.

```
-- forward citations of a seed patent (sketch, untested)
SELECT c.publication_number
FROM `patents-public-data.google_patents_research.publications` r,
     UNNEST(r.cited_by) c
WHERE r.publication_number = 'US-787412-A'
```

Cost: BigQuery on-demand gives the first 1 TiB of query data per month free, then $6.25/TiB (pricing page, Oct 13). The table doc lists `patents.publications` at 899 GB (a 2018 snapshot): select only needed columns, filter by CPC, and dry-run every query for its byte count. UNVERIFIED current table size and refresh schedule; whether Chew's Google account can query without billing set up. **Fallback:** patent-page citations only (no BigQuery).

Legal status and expiry

Show Google's status text and the dates; compute “likely expired” only as a flag (US utility patents generally run 20 years from filing, extended or cut short by fees and terms). Every status line carries “not legal advice; check with a patent attorney”. Freedom-to-operate is out of scope.

💡 Wildcard: tech NEW PLANNED

1. **Strip to principle** (Super): remove the product and industry words; keep the physics and the numbers. Exhaust → “smooth pressure pulses in a flowing gas, 100 Hz, ~400 °C”.
2. **Pick far niches**: CPC sections and classes away from home (e.g. G10D wind instruments, A61F 2/24 heart valves, toys, kitchen tools), plus non-patent sources for nature / biomimicry and biology (Tavily, arXiv).
3. **Search** each niche with the principle's words (patent-speak per niche), take the top 2–3 mechanisms.
4. **Transfer** (Super): “borrow X: here's how it maps” + a sizing problem (e.g. quarter-wave length = c / 4f).
5. **Does the math work?** The sizing runs in the sandbox (Python = record) and through the OpenMath checker; result vs the team's constraints (size, temperature, cost).
6. **Rank** by `surprise × math_fit`.

surprise	CPC distance from the home class: same subclass 0 · same class 0.25 · same section 0.5 · other section or non-patent 1.0; ×0.25 if the same mechanism already appears in the home-class hits (e.g. Helmholtz resonators sit in F01N 1/023)
math_fit	1 = sizes inside constraints · 0.5 = works but stretches one · 0 = numbers don't work (listed under “fun, but no”)

Calculators: v1 uses sandboxed model-written SymPy; add vetted `quarter_wave_tube` and `helmholtz` calculators for the demo problem if time allows. Weights and thresholds are first guesses.

Data model and JSON

Everything the UI shows comes from one `Report` (pydantic, `schema.py`). Planned fields are marked `// PLANNED`.

```
Report { input_claim, mode, model, topic, summary,
  sources[Source], verdicts[ClaimVerdict], concept, overall,
  spec: ProductSpec, gaps[GapAnalysis], archive_hits[ArchiveHit],
  math_checker: MathCheckerInfo,
  search_plan: SearchPlan|null }          // PLANNED
ClaimVerdict { claim, calc, verdict, explanation,
  guardrail_note, math_check: MathCheck }
ProductSpec { design, title, requirements, headline[],
  specs[{name,value,unit,basis}], series, safety[], bom[],
  bom_total_usd, render_prompt, caveats[], math_check }
GapAnalysis { claim_id, missing_piece, requirements[
  {statement, target, feasibility, leads[], evidence,
   score, rank}] }
MathCheck { target, status, headline, source, problem,
  steps[], key_equations[], checker_value, python_value,
  rel_diff, tolerance, symbolic_match }
SearchPlan {                              // PLANNED
  mode: "find"|"check"|"improve", input{goal, failed[],
    idea, constraints[], last_plan_id},
  abstraction{function, conflict{improve, without},
    principle, exclude[]},
  terms[{term, synonyms[], patent_speak[]}],
  cpc[{code, title, role:"home"|"cross", why}],
  queries[{engine:"gpatents_link"|"tavily"|"bigquery", text}],
  hits[{pub, title, date, cpc[], found_via, claim_quote,
    fit, status_text, status_flag, url, rank}],
  wildcard[{niche, cpc|null, mechanism, transfer,
    sizing{problem, value, unit, math_check}, surprise,
    math_fit, score}],
  trail[{step, did, kept, dropped}],
  diff{new[], dropped[], reranked[]} }    // Improve mode
```

Models and cost

Job	Model	Note
Extract, design pick, planner steps 1–3 and 5, Wildcard transfer	`nvidia/nemotron-3-super-120b-a12b`	Lead; Token Factory
Verdict text, gap queries	`nvidia/Nemotron-3_5-Lightning`	Fast, cheap
Math derivations + cross-check	`nvidia/OpenMath-Nemotron-7B`	Self-hosted vLLM, CC-BY-4.0
Embeddings / render	`/v1/embeddings · flux-schnell`	UNVERIFIED as served

Token Factory model ids come from Nebius' Aug 2026 deprecation notice; UNVERIFIED on Chew's key until `scripts/check_nebius.py` runs. No model does arithmetic: it picks a calculator or writes code.

Verdict run	~40k in / 8k out tokens, 3–4 Tavily searches + extract; ~45–135 s EST.
Planner run PLANNED	4–6 Super calls, 6–12 Tavily searches, 10–30 patent pages, 0–3 BigQuery queries; 1–3 min EST.
Wildcard	+2 Super calls, 4–8 searches, 3–6 sizing checks (OpenMath in parallel) EST.
OpenMath GPU	≈ $1.55/h on an L40S, ≈ 64 h for $100 EST.; run it only while working

Nemotron 3 Super price: third-party listing only UNVERIFIED. Cache every live run; the demo replays from cache.

Errors and fallbacks

Model contradicts the math	Guardrail overrules it BUILT
Calc fails / times out	Capped at MISSING PIECE BUILT
Nemotron bad JSON	json_schema → json_object → text parse BUILT
OpenMath down / no endpoint	Checker “skipped” or “error”; run continues BUILT
BigQuery unavailable	Citations from patent pages only PLANNED
Claim quote not on page	Fit set to 0, hit marked “unconfirmed” PLANNED
Wildcard math fails	Idea kept under “fun, but no” PLANNED
Tavily down / demo over budget	Archive-only run; replay mode PLANNED

Testing and security

- BUILT 82 tests pass, 1 skipped (optional embeddings library) from a fresh unzip, no keys: calculators, sandbox escapes, designers, spec sheets, gap ranking, archive, eval, math checker (mocked client, cross-check, skip path), mock pipeline, UI.
- BUILT 20-claim eval (5 WORKS, 6 MISSING PIECE, 9 BREAKS PHYSICS) + chatbot baseline; real accuracy needs `run_eval.py --live --baseline`.
- PLANNED Planner: mocked exhaust run end to end; CPC validator; quote-match; surprise math; robots.txt-safe fetcher (only `/patent/` URLs).
- Keys in `.env` only; pages and PDFs are data, never instructions; model code only in the sandbox; the OpenMath API sits behind an SSH tunnel by default.

Mapping to the judging criteria

Criterion	Where Tesla Lab scores

Technological implementation	Two NVIDIA models with clear roles (Super leads, OpenMath derives) cross-checked against Python; guardrails; designers under a safety limit; page-cited RAG; eval vs chatbot; planner with CPC, citation chains and claim-quote checks
Design (complete product)	One input box, live progress, verdict cards, spec card + PDF, roadmap, math-checker badges; planner modes with a visible reasoning trail
Potential impact	R&D teams get unstuck in minutes; makers get buildable specs; teachers and journalists get checked math and primary sources
Quality of the idea	Not a debunker: it rescues the working part, designs it, and hunts the missing piece across industries. 💡 Wildcard is the move experts miss
Rules / Tavily bonus	Token Factory runtime calls ✓ · NVIDIA open models ✓ · video ≤ 3 min · working demo. Tavily on every run: research, gap search, and patent discovery

Build schedule · Oct 14–30

Day	Work
Wed 14	Keys. `check_nebius.py` → lock model ids; first live run. GPU VM up, `check_openmath.py`, stop VM
Thu 15	`run_eval.py --live --baseline` → real accuracy; fix prompts
Fri 16	**Planner 1:** SearchPlan schema; Abstract + Expand (Super JSON); mock exhaust run
Sat 17	**Planner 2:** CPC table + validator, cross-industry match; Google Patents query links; Tavily patent discovery
Sun 18	**Planner 3:** patent-page reader (claims, citations, status); Chain; Read-claims fit + quote check
Mon 19	**Planner 4:** S9 UI (mode tabs, trail, hit cards); Find + Check modes; tests
Tue 20	**Wildcard 1:** principle, far-niche search, transfer
Wed 21	**Wildcard 2:** sizing via sandbox + OpenMath (GPU on); surprise × math ranking; S10 cards
Thu 22	Improve mode (constraint list, re-run diff, “new since” date filter). MISSING PIECE → “Search patents for this”
Fri 23	Buffer. Optional: BigQuery citation chain; FBI/CIA PDFs in the archive
Sat 24	Deploy public demo (Nebius VM + Streamlit Cloud backup), replay mode, rate limits
Sun 25	Buffer; outside feedback (KL / Singapore event if it fits)
Mon 26	Lock demos: 3 Tesla claims + 1 exhaust planner run; run live 10×; cache best

Tue 27	Draft video (≤ 3:00); fix what looks weak
Wed 28	Final video + Devpost text (Token Factory, Nemotron, OpenMath, Tavily)
Thu 29	**Submit**, a day early. Test the link from a phone
Fri 30	Deadline 10:00am PDT = Sat Oct 31, 1:00am Malaysia. Hands off

If time runs short, cut in this order: (1) BigQuery: use patent-page citations only · (2) Improve mode: show it as a roadmap slide · (3) Check mode as its own tab: keep Find + Wildcard · (4) vetted acoustic calculators: sandbox code only · (5) FBI/CIA documents, Nebius Sandboxes, shareable report page.
Never cut: Find mode with the 5-step trail, the 💡 Wildcard with one computed number, and the existing verdict + Design Engine + math-checker demo. GPU: run the VM on Oct 14, 21 and 26–27 only (~20–30 h **EST.**) to keep the $100 credit safe.

What's verified, what isn't

Checked

- Repo v3: 82 tests pass, 1 skipped, from a fresh unzip in mock mode; Tesla Shelf, gap and demo numbers on these pages come from the repo.
- OpenMath-Nemotron ids, licence, prompt format and temperature (Hugging Face, NeMo-Skills docs); Nebius VM commands, L40S/H100 prices, “Linux shutdown keeps billing” (Nebius docs, Oct 13). ICNIRP values (PDFs).
- Google Patents query syntax: `cpc:`, `CPC=…/low`, child codes included, TI/AB/CL fields, before/after, country, status, NEAR/ADJ (ranking only), non-patent literature, Prior Art Finder (Help → Searching, Oct 13).
- Patent pages show claims, CPC, Patent Citations, Cited By, Similar Documents and legal status with Google's disclaimer (US787412A, Oct 13). robots.txt allows `/patent/` and disallows the rest.
- Google Patents Public Datasets on BigQuery exist (2017 Google Cloud blog; GitHub table docs list `citation`, `cited_by`, `cpc`, claims). BigQuery free tier 1 TiB/month (pricing page). Tavily `include_domains` (docs).
- CPC titles: F01N 1/02, 1/023, 1/06; F16L 55/04; G10K 11/16, 11/172; A61F 2/24; G10D 7/00 (USPTO CPC pages). Wildcard sizes: computed in Python with the stated assumptions.

Not verified

- Any live call: Nemotron ids on Chew's key, JSON mode, Tavily with his key, OpenMath on a real GPU (mock derivations are canned), deploy scripts on Nebius.
- Planner and Wildcard: not built; ranking weights, timings and costs are guesses; Tavily's hit quality on patent pages.
- BigQuery: current table size and freshness, the SQL sketch, whether queries work without billing set up.
- Whether the $100 credit covers disk/IP; Nemotron prices; BOM prices; design assumptions.
- Gap-search leads are snippets, not read papers. Legal status is Google's assumption; nothing here is legal advice.

Next step for Chew: add the Nebius + Tavily keys, run `check_nebius.py`, then `run_eval.py --live --baseline`. Then start the planner on Fri Oct 16.